Lecture Notes: k-Nearest Neighbors

CIC1205 - Machine Learning

Eduardo Bezerra

March 4, 2026

0 Contents

1 Introduction **2**

2 Geometric Interpretation of a Dataset **3**
2.1 Distance Matrix and Heatmaps . . . 3

3 Definition of Neighbors **4**
3.1 Choosing a Distance Metric . . . 4

4 k-NN for Classification **5**
4.1 Steps of the Algorithm (Inference Phase) . . . 5
4.1.1 Probabilistic Interpretation . . . 6
4.1.2 Choosing k . . . 6
4.1.3 A Worked Example . . . 7
4.2 Implementation in Scikit-Learn . . . 8

5 k-NN for Regression **9**
5.1 Steps of the Algorithm (Inference Phase) . . . 9
5.1.1 Geometric Interpretation . . . 10
5.1.2 Choosing k . . . 10
5.2 A Worked Example . . . 11
5.2.1 Step 1: Compute Distances . . . 11
5.2.2 Step 2: Select the k Nearest Neighbors . . . 11
5.2.3 Step 3: Average Target Values and Predict . . . 12
5.3 Implementation in Scikit-Learn . . . 12

6 k-NN: Inductive Bias **13**

7 Final Remarks **13**
7.1 Decision Boundaries Can Be Complex . . . 13
7.2 Computational Considerations . . . 13
7.3 High Dimensionality and the Curse of Dimensionality . . . 13

8 Summary **14**

1 Introduction

The k**-Nearest Neighbors** (k-NN) algorithm is one of the simplest and most widely used methods in Data Mining and Machine Learning. Despite its simplicity, it often produces satisfactory results and can be applied to both:

- **Classification** (predict a discrete label), and
- **Regression** (predict a continuous value).

A defining characteristic of k-NN is that it is a **lazy learning** algorithm:

- during "training", it does not fit a parametric model;
- it essentially **stores** the training examples;
- predictions are computed later, at inference time, by comparing a query point to the stored examples.

k-Nearest Neighbors (k-NN)

Given a query example $\mathbf{x}$, k-NN finds the k *closest* training examples according to a distance (or similarity) measure, and predicts the output for $\mathbf{x}$ based on those neighbors.

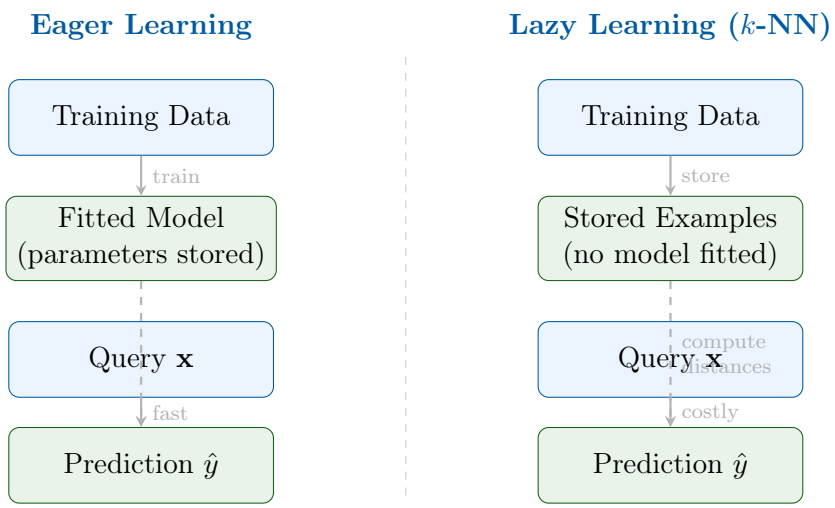

Figure 1: Eager vs. lazy learning. An eager learner (e.g., decision tree) fits a model during training and predicts cheaply. A lazy learner (k-NN) defers all computation to inference time, making training trivial but prediction more costly.

Entry Ticket 1

Why is k-NN called **lazy learning**? In what sense does it "learn" at inference time rather than training time?

2 Geometric Interpretation of a Dataset

Many supervised learning algorithms (including k-NN) interpret each example as a **point in a geometric space**.

Let $\mathbf{x} \in \mathbb{R}^d$ be a feature vector with d predictive features. Then each training example corresponds to a point in a d-dimensional feature space.

Why geometry helps

If examples are points, then **distance** becomes a natural proxy for **similarity**: the closer two points are, the more similar the corresponding examples are (under the chosen metric).

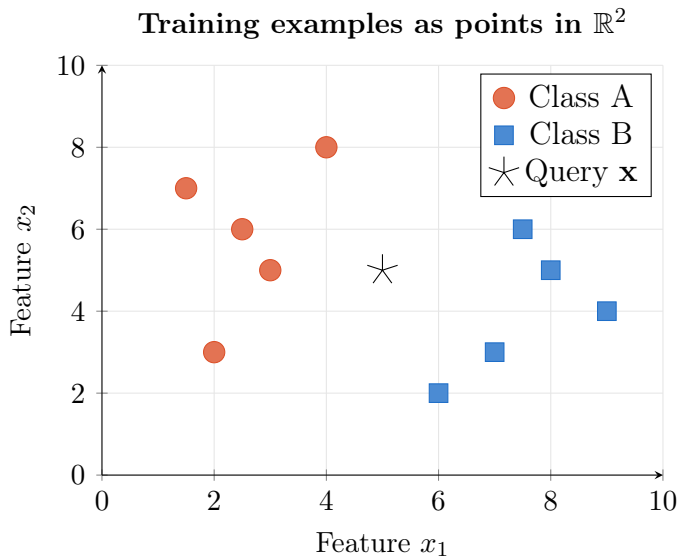

Figure 2: A 2D dataset: each example is a point in $\mathbb{R}^2$. Class A (circles) clusters in the left region; Class B (squares) clusters in the right region. The query point $\mathbf{x}$ (star) will be classified based on which class its nearest neighbors belong to.

2.1 Distance Matrix and Heatmaps

Given a dataset with examples $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}$, we may compute a **distance matrix** $D \in \mathbb{R}^{m \times m}$ where

$$D_{ij} = \mathrm{dist}\big(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\big).$$

A heatmap visualization of D can help us inspect which points are close/far, and sometimes reveal clusters or outliers.

Entry Ticket 2

Suppose a dataset contains two identical examples $\mathbf{x}^{(i)} = \mathbf{x}^{(j)}$ with $i \neq j$. What value should appear in the distance matrix entry D_{ij}? What would this look like in a heatmap?

3 Definition of Neighbors

Let $X = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$ be the training set and let $\mathbf{x}$ be a query point.

Neighborhood

Given an integer $k \geq 1$, the **k-neighborhood** of $\mathbf{x}$ is the set of the k training examples with smallest distance to $\mathbf{x}$.

3.1 Choosing a Distance Metric

A key design choice in k-NN is the distance (or similarity) measure.

Common metrics (continuous features)

For $\mathbf{x}, \mathbf{z} \in \mathbb{R}^d$:

- **Manhattan (L1)**:

$$d_1(\mathbf{x}, \mathbf{z}) = \sum_{j=1}^{d} |x_j - z_j|.$$

- **Euclidean (L2)**:

$$d_2(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2}.$$

- **Cosine similarity** (often used for sparse/high-dimensional vectors):

$$\cos(\mathbf{x}, \mathbf{z}) = \frac{\mathbf{x} \cdot \mathbf{z}}{\|\mathbf{x}\| \, \|\mathbf{z}\|}.$$

Common metrics (discrete features)

- **Hamming distance** (counts positions that differ).
- Other domain-specific measures (e.g., value distance measures).

Feature scaling matters

Because k-NN relies directly on distances, features with larger numerical ranges can dominate the metric. Therefore, **standardization** (e.g., z-score scaling) is often crucial for good performance.

Entry Ticket 3

Consider a dataset with two features: `age` (0–100) and `annual_income` (0–500,000). If we apply Euclidean distance without scaling, which feature is likely to dominate the distances? Why?

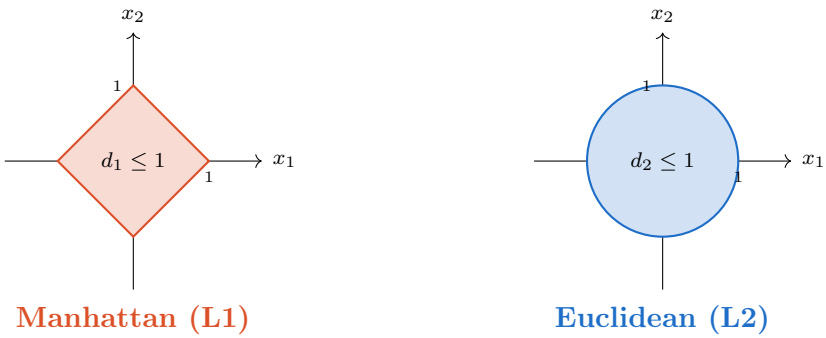

Figure 3: Unit balls for Manhattan (L1, diamond shape) and Euclidean (L2, circle) distances in $\mathbb{R}^2$. The geometry of "closeness" differs between metrics, which can meaningfully change which neighbors are selected.

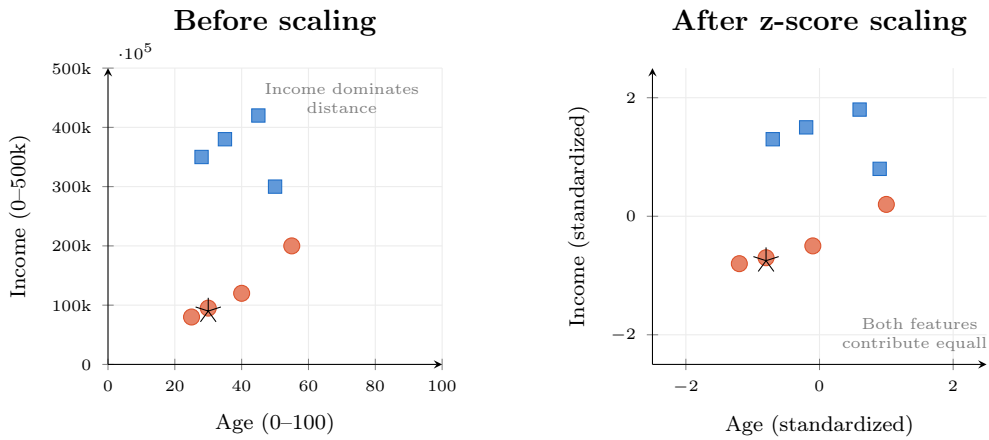

Figure 4: Effect of feature scaling. Before scaling (left), the income axis spans 500k units while age spans only 100; income dominates any distance computation. After z-score standardization (right), both features contribute equally to the distance.

4 k-NN for Classification

4.1 Steps of the Algorithm (Inference Phase)

In its standard classification form, k-NN proceeds as follows for a query $\mathbf{x}$:

1. Compute the distances from $\mathbf{x}$ to all training examples.
2. Select the k nearest training examples (the neighborhood).
3. Count how many neighbors belong to each class.
4. Predict the class by **majority vote**.

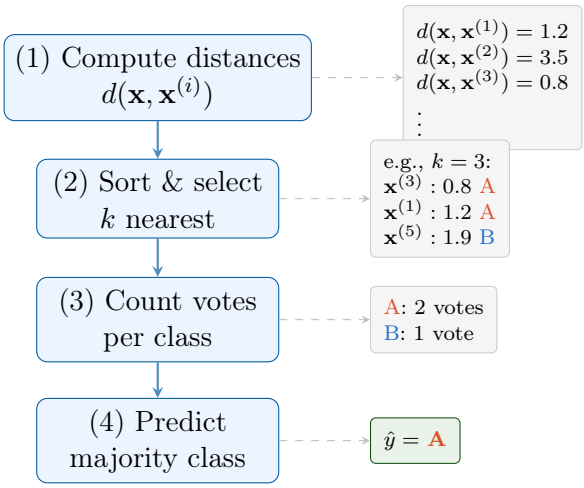

Figure 5: The four steps of k-NN inference ($k = 3$, binary classification). Each step is illustrated with a concrete example: three distances are computed, the three nearest neighbors contain 2 votes for A and 1 for B, so A is predicted.

4.1.1 Probabilistic Interpretation

Let the possible classes be $\{c_1, \ldots, c_K\}$. Let $q(c_i)$ be the number of neighbors among the k nearest that belong to class c_i. Then a natural estimate is:

$$\widehat{\Pr}(c_i \mid \mathbf{x}) = \frac{q(c_i)}{k}. \tag{1}$$

Majority Rule

The predicted label is

$$\hat{y}(\mathbf{x}) = \arg\max_{c_i} \widehat{\Pr}(c_i \mid \mathbf{x}) = \arg\max_{c_i} q(c_i).$$

Entry Ticket 4

Suppose $k = 5$ and the neighborhood contains 3 examples of class `A` and 2 examples of class `B`.

1. Compute $\widehat{\Pr}(\texttt{A} \mid \mathbf{x})$ and $\widehat{\Pr}(\texttt{B} \mid \mathbf{x})$.
2. What class is predicted by majority vote?

4.1.2 Choosing k

There is no universally optimal k:

- small k can lead to **high variance** (sensitive to noise);
- large k can lead to **high bias** (over-smoothing boundaries).

In practice, k is chosen via **model selection** (e.g., validation).

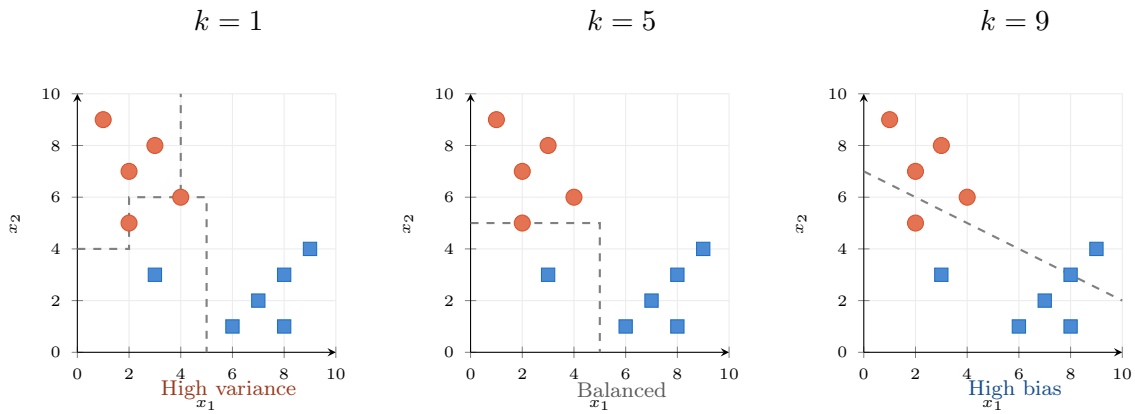

Figure 6: Effect of k on the decision boundary. For $k = 1$ (left), the boundary is highly irregular and follows individual training points (high variance). For $k = 9$ (right), the boundary becomes very smooth but may ignore real structure (high bias). A moderate $k = 5$ (center) often offers a good balance.

Entry Ticket 5

Intuitively, what happens to the decision boundary when k increases from 1 to a much larger value? Relate your answer to bias and variance.

4.1.3 A Worked Example

This section illustrates the full k-NN inference procedure on a small concrete dataset.

Dataset Consider the following 7 training examples with two continuous features x_1 (hours studied) and x_2 (hours slept), and binary class label `Pass`/`Fail`:

Example	x_1 (study)	x_2 (sleep)	**Label**
$\mathbf{x}^{(1)}$	1	4	Fail
$\mathbf{x}^{(2)}$	2	6	Fail
$\mathbf{x}^{(3)}$	3	7	Pass
$\mathbf{x}^{(4)}$	5	5	Pass
$\mathbf{x}^{(5)}$	6	8	Pass
$\mathbf{x}^{(6)}$	7	3	Fail
$\mathbf{x}^{(7)}$	8	7	Pass
Query $\mathbf{x}$	4	6	**?**

Step 1: Compute Euclidean Distances We compute $d_2(\mathbf{x}, \mathbf{x}^{(i)}) = \sqrt{(4 - x_1^{(i)})^2 + (6 - x_2^{(i)})^2}$ for each training example:

Example	Label	Computation	Distance
$\mathbf{x}^{(1)}$	Fail	$\sqrt{(4-1)^2+(6-4)^2}=\sqrt{9+4}$	3.61
$\mathbf{x}^{(2)}$	Fail	$\sqrt{(4-2)^2+(6-6)^2}=\sqrt{4+0}$	2.00
$\mathbf{x}^{(3)}$	Pass	$\sqrt{(4-3)^2+(6-7)^2}=\sqrt{1+1}$	1.41
$\mathbf{x}^{(4)}$	Pass	$\sqrt{(4-5)^2+(6-5)^2}=\sqrt{1+1}$	1.41
$\mathbf{x}^{(5)}$	Pass	$\sqrt{(4-6)^2+(6-8)^2}=\sqrt{4+4}$	2.83
$\mathbf{x}^{(6)}$	Fail	$\sqrt{(4-7)^2+(6-3)^2}=\sqrt{9+9}$	4.24
$\mathbf{x}^{(7)}$	Pass	$\sqrt{(4-8)^2+(6-7)^2}=\sqrt{16+1}$	4.12

Step 2: Select the k Nearest Neighbors Sorting by distance and selecting the $k=3$ nearest:

Rank	Example	Label
1	$\mathbf{x}^{(3)}$ (dist $=1.41$)	**Pass**
2	$\mathbf{x}^{(4)}$ (dist $=1.41$)	**Pass**
3	$\mathbf{x}^{(2)}$ (dist $=2.00$)	**Fail**

Step 3: Majority Vote and Prediction Counting votes:

$$q(\texttt{Pass})=2, \qquad q(\texttt{Fail})=1.$$

Estimated probabilities (Equation (1)):

$$\widehat{\Pr}(\texttt{Pass}\mid\mathbf{x})=\frac{2}{3}\approx 0.67, \qquad \widehat{\Pr}(\texttt{Fail}\mid\mathbf{x})=\frac{1}{3}\approx 0.33.$$

The prediction is $\hat{y}(\mathbf{x})=$ **Pass**.

4.2 Implementation in Scikit-Learn

Scikit-Learn provides `KNeighborsClassifier` to implement k-NN for classification. The most important hyperparameter is:

- `n_neighbors` (the value of k).

Other commonly used settings include:

- `metric`: which distance metric to use (e.g., Euclidean, Manhattan).
- `weights`:
  - `uniform`: all neighbors count equally;
  - `distance`: closer neighbors count more (distance-weighted voting).

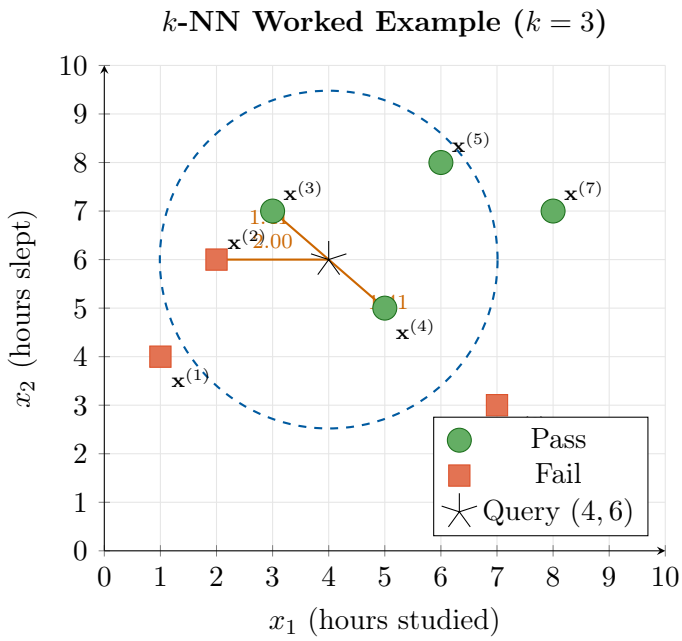

Figure 7: Visualization of the worked example with $k = 3$. The dashed circle encompasses the 3 nearest neighbors of the query point (star). Orange lines show the distances to the neighbors: $\mathbf{x}^{(3)}$ and $\mathbf{x}^{(4)}$ (Pass) and $\mathbf{x}^{(2)}$ (Fail). Majority vote yields **Pass**.

Train–test workflow

A typical workflow is:

1. Split data into train/test (or train/validation/test).
2. Fit the classifier on the training set (stores data internally).
3. Use `predict` to classify new examples.
4. Use `predict_proba` to inspect class probabilities.

Entry Ticket 6

What is the practical difference between calling `predict` and `predict_proba` on a trained k-NN classifier? When might probabilities be more useful than hard labels?

5 k-NN for Regression

When k-NN is used for regression, we still find the k nearest neighbors, but the prediction is computed from their **target values** rather than class votes.

5.1 Steps of the Algorithm (Inference Phase)

In its regression form, k-NN proceeds as follows for a query $\mathbf{x}$:

1. Compute the distances from $\mathbf{x}$ to all training examples.
2. Select the k nearest training examples (the neighborhood).
3. Collect the target values $y^{(1)}, \ldots, y^{(k)}$ of the neighbors.

4. Predict the **average** of those target values.

k-NN Regression Prediction

Let the k nearest neighbors of $\mathbf{x}$ have target values $y^{(1)}, \ldots, y^{(k)}$. The standard prediction is the mean:

$$\hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{j=1}^{k} y^{(j)}. \tag{2}$$

Regression Prediction Rule

Unlike classification (which takes a majority vote over discrete labels), regression takes the **arithmetic mean** of the neighbors' continuous target values. The result is a piecewise-constant function over the feature space.

5.1.1 Geometric Interpretation

As $\mathbf{x}$ moves continuously through the feature space, the set of its k nearest neighbors changes at certain boundaries. Within each region where the neighborhood is fixed, $\hat{y}(\mathbf{x})$ is a constant (the mean of those k targets). The overall prediction function is therefore a **step function** — piecewise constant with discontinuities at the boundaries between neighborhood regions.

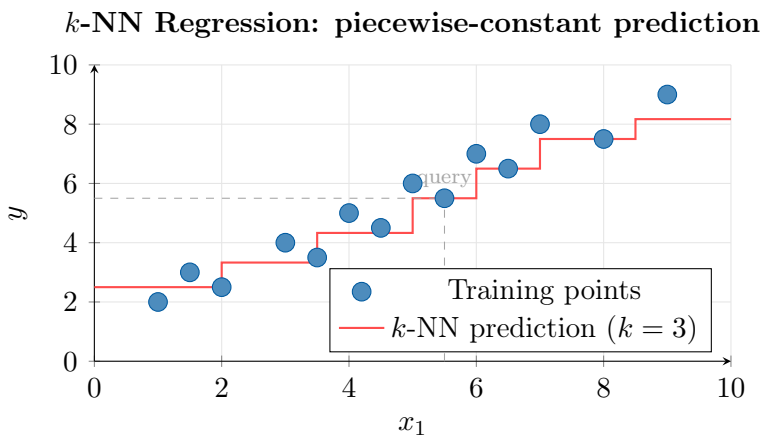

Figure 8: k-NN regression with $k = 3$. For any query x_1, the prediction is the average of the target values of the 3 nearest training points. The result is a piecewise-constant function; as k increases, the steps become wider and the curve smoother.

5.1.2 Choosing k

As in classification, the choice of k controls the bias–variance trade-off:

- **Small** k (e.g., $k = 1$): the prediction closely follows individual training points, leading to **high variance** and potential overfitting.
- **Large** k: the prediction averages over many points, producing a smoother but possibly **over-smoothed** function (high bias).

Entry Ticket 7

Why does averaging neighbor targets (Equation (2)) make sense as a regression strategy? What kind of assumption about the target function does this encode?

5.2 A Worked Example

Consider the following 6 training examples with one feature x_1 and a continuous target y:

Example	x_1	y
$\mathbf{x}^{(1)}$	1.0	2.0
$\mathbf{x}^{(2)}$	2.0	3.5
$\mathbf{x}^{(3)}$	3.0	2.8
$\mathbf{x}^{(4)}$	5.0	6.1
$\mathbf{x}^{(5)}$	6.0	5.4
$\mathbf{x}^{(6)}$	8.0	8.0
Query $\mathbf{x}$	4.0	**?**

5.2.1 Step 1: Compute Distances

Since there is a single feature, the Euclidean distance reduces to $d_2(\mathbf{x}, \mathbf{x}^{(i)}) = |4.0 - x_1^{(i)}|$:

Example	**Computation**	**Distance**
$\mathbf{x}^{(1)}$	$\lvert 4.0 - 1.0 \rvert$	3.0
$\mathbf{x}^{(2)}$	$\lvert 4.0 - 2.0 \rvert$	2.0
$\mathbf{x}^{(3)}$	$\lvert 4.0 - 3.0 \rvert$	1.0
$\mathbf{x}^{(4)}$	$\lvert 4.0 - 5.0 \rvert$	1.0
$\mathbf{x}^{(5)}$	$\lvert 4.0 - 6.0 \rvert$	2.0
$\mathbf{x}^{(6)}$	$\lvert 4.0 - 8.0 \rvert$	4.0

5.2.2 Step 2: Select the k Nearest Neighbors

Sorting by distance and selecting the $k = 3$ nearest:

Rank	**Example**	**Distance**	**Target y**
1	$\mathbf{x}^{(3)}$	1.0	2.8
2	$\mathbf{x}^{(4)}$	1.0	6.1
3	$\mathbf{x}^{(2)}$	2.0	3.5

5.2.3 Step 3: Average Target Values and Predict

Applying Equation (2):

$$\hat{y}(\mathbf{x}) = \frac{1}{3}\big(2.8 + 6.1 + 3.5\big) = \frac{12.4}{3} \approx 4.13.$$

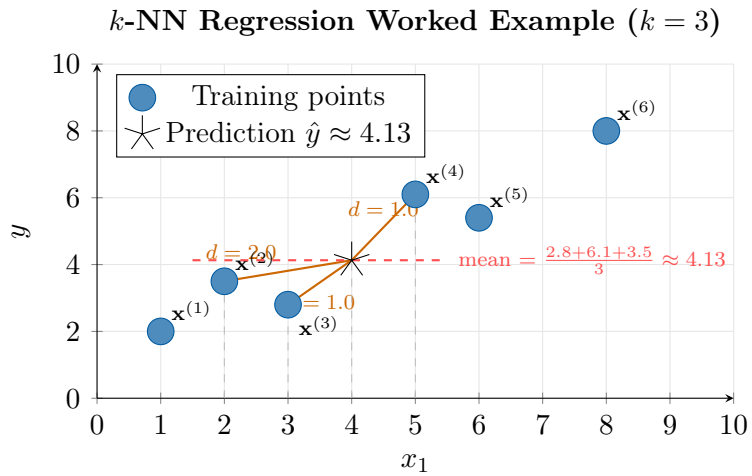

Figure 9: Regression worked example with $k = 3$. The 3 nearest neighbors of the query $x_1 = 4.0$ are $\mathbf{x}^{(3)}$, $\mathbf{x}^{(4)}$, and $\mathbf{x}^{(2)}$ (orange lines). The prediction is the average of their target values, shown as the dashed red line at $\hat{y} \approx 4.13$.

5.3 Implementation in Scikit-Learn

Scikit-Learn implements k-NN regression via `KNeighborsRegressor`, with the same key hyperparameters as the classifier:

- `n_neighbors` (k): number of neighbors.
- `metric`: distance function (default: Euclidean).
- `weights`: `uniform` (equal weights) or `distance` (closer neighbors contribute more to the average).

Distance-weighted regression

With `weights='distance'`, the prediction becomes a **weighted mean**:

$$\hat{y}(\mathbf{x}) = \frac{\sum_{j=1}^{k} w_j\, y^{(j)}}{\sum_{j=1}^{k} w_j}, \quad w_j = \frac{1}{d(\mathbf{x},\, \mathbf{x}^{(j)})}.$$

This gives more influence to closer neighbors and often reduces prediction error near sharp transitions in the target function.

Entry Ticket 8

In the worked example above, recompute the prediction using distance-weighted averaging (i.e., $w_j = 1/d_j$) for the same 3 neighbors. Does the result differ from the unweighted mean ≈ 4.13? In which direction, and why?

6 k-NN: Inductive Bias

The **inductive bias** of a learning algorithm is the set of assumptions it uses to generalize from finite data.

For k-NN, the central assumption is:

Inductive Bias of k-NN

Examples that are close in feature space tend to have similar outputs. Equivalently, the target function is assumed to be **locally smooth / locally constant**.

This is:

- local smoothness of class labels for classification; and
- local smoothness of real-valued targets for regression.

Entry Ticket 9

Give a real-world scenario where the inductive bias of k-NN is reasonable. Then give a scenario where it might fail (i.e., nearby points often have different labels).

7 Final Remarks

7.1 Decision Boundaries Can Be Complex

Despite being conceptually simple, k-NN can represent very complex decision boundaries, especially for small values of k. For $k = 1$, the resulting partition is related to the **Voronoi diagram** induced by the training points: each training point "owns" the region of space closer to it than to any other training point.

7.2 Computational Considerations

Because k-NN compares a query to many (or all) training points, inference can be costly:

- **Training time**: usually small (store the data).
- **Prediction time**: can be large (distance computations).

7.3 High Dimensionality and the Curse of Dimensionality

In high-dimensional feature spaces, distances can become less informative. This is a manifestation of the **curse of dimensionality**: as d grows, the volume of space increases

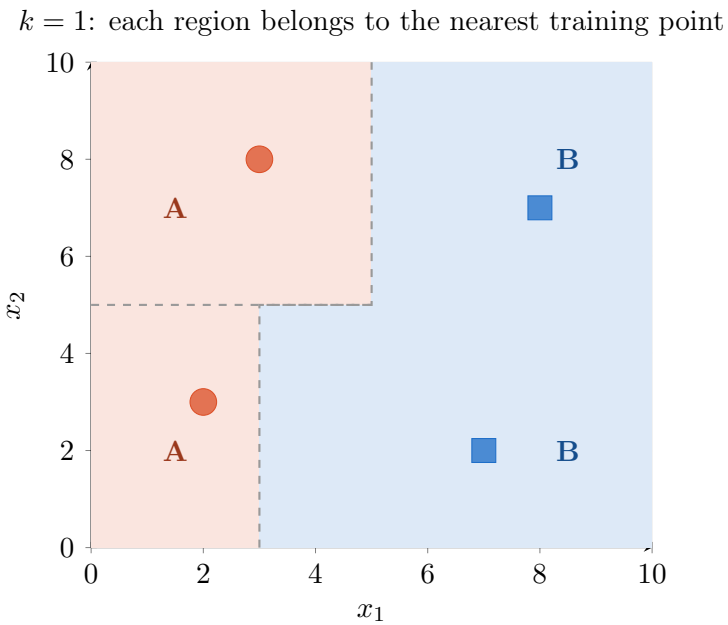

Figure 10: For $k = 1$, the feature space is partitioned into regions where each region takes the label of its nearest training point (Voronoi-like structure). The boundary between regions can be highly irregular, following individual training points.

exponentially, and training points become increasingly sparse. As a consequence, the distances between any two points tend to become very similar; there is no longer a meaningful notion of "close" vs. "far". This may degrade the performance of nearest-neighbor methods unless feature engineering, dimensionality reduction, or appropriate similarity measures are used.

Entry Ticket 10

Why might k-NN performance degrade in very high dimensions? Provide an intuitive explanation that relates to how distances behave.

8 Summary

Concept	**Key idea / formula**
Core idea	predict using the k nearest stored training examples
Lazy learning	little/no parametric training; heavy inference
Classification probability	$\widehat{\Pr}(c_i \mid \mathbf{x}) = q(c_i)/k$
Classification rule	$\hat{y}(\mathbf{x}) = \arg\max_{c_i} q(c_i)$
Regression rule	$\hat{y}(\mathbf{x}) = \frac{1}{k}\sum_{j=1}^{k} y^{(j)}$
Key hyperparameter	k (`n_neighbors`)
Important practical issue	feature scaling (distance sensitivity)

Distance concentration: max/min distance ratio (100 random points, uniform $[0,1]^d$)

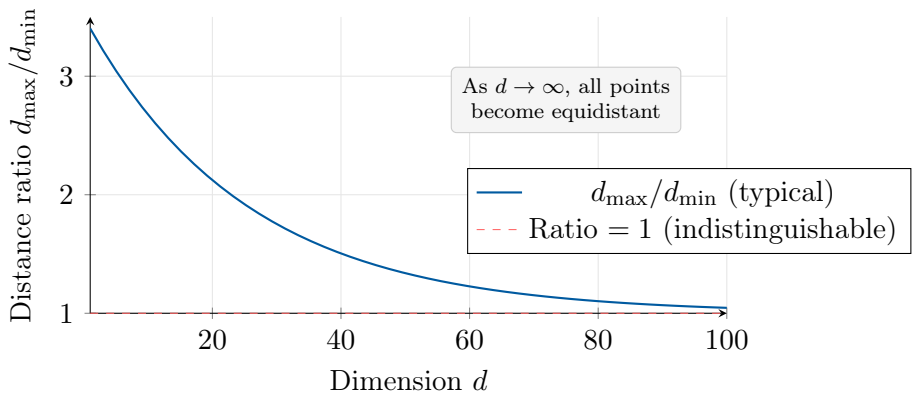

Figure 11: Illustration of distance concentration with increasing dimensionality. As d grows, the ratio between the maximum and minimum pairwise distance approaches 1: all points appear equally far from each other, making the notion of "nearest neighbor" unreliable.

Key Takeaways

1. k-NN is simple, flexible, and often effective for both classification and regression.
2. It relies on a notion of similarity, so the choice of distance metric and feature scaling are critical.
3. For classification, prediction is based on majority voting and can be interpreted probabilistically.
4. The value of k controls the bias–variance trade-off and is typically chosen by validation.
5. Although training is cheap, inference can be expensive, especially for large datasets and high dimensions.